

Grado en Ingeniería Informática

Computabilidad y Algoritmia

REPASO

El entorno de trabajo IaaS para la asignatura

Práctica no evaluable

Semana del 7 al 11 de septiembre de 2026

Esta práctica consta de dos partes. En primer lugar, deberá realizar una serie de ejercicios a través de la infraestructura IaaS que utilizó durante el curso pasado. En segundo lugar, realizará una práctica de programación en C++ de forma remota. A modo de recordatorio, en el apartado 3 de este documento encontrará toda la información necesaria para utilizar la infraestructura IaaS.

Para realizar la entrega de la práctica a través del aula virtual, deberá comprimir en un único archivo tanto el documento con los ejercicios del IaaS como el código fuente del ejercicio de programación en C++. El archivo comprimido deberá tener el nombre `Apellido1Apellido2NombreCyAP0IaaS.tgz`.

1. Entorno IaaS

El documento PDF deberá contener:

1. Su nombre y apellidos.
2. Su dirección de correo electrónico institucional.
3. La dirección IP de su máquina virtual en la asignatura.

Además de esta información, el documento deberá incluir cuatro imágenes (capturas de pantalla de su ordenador) que acrediten la realización de algunos de los ejercicios propuestos en la práctica. En particular, deberá incluir las siguientes:

1. Una captura de pantalla del perfil de su cuenta en GitHub en la que se muestren su nombre y el nombre de usuario de esa cuenta.

2. Una captura de pantalla completa en la que se muestre que ha accedido a su máquina mediante un navegador y la consola (VNC Browser).
3. Una captura de pantalla de VS Code que muestre que está editando el fichero `pell.cpp`, alojado en su máquina virtual IaaS, que contendrá el código correspondiente al ejercicio de programación del segundo apartado.
4. Una captura de pantalla en la que se muestre una terminal de Linux desde la que se haya establecido una conexión SSH con su máquina virtual IaaS utilizando un comando como `ssh mi_maquina_ibasica`. La imagen deberá mostrar la ejecución de los siguientes comandos:

```
hostname
whoami
ssh mi_maquina_ibasica
hostname
whoami
```

2. Práctica de programación

1. Utilizando una conexión remota con su máquina virtual y el VSC, codifique en C++ un programa denominado `pell.cc` que, al ejecutarse, tome como parámetro de línea de comandos un número entero N y que, como resultado, imprima en pantalla los N primeros términos de la [serie de Pell](#) [1].

Cuando termine la edición y depuración de su programa, abra un terminal en Linux, conecte por ssh con su VM y compile y ejecute el programa usando comandos Linux para ello.

Cuando el programa se ejecuta como:

```
$ ./pell 8
```

La salida impresa en pantalla debería ser:

```
0 1 2 5 12 29 70 169
```

3. Recordatorio: GNU/Linux en su ordenador de trabajo

1. Para el trabajo en las prácticas de la asignatura se utilizará intensivamente el Sistema Operativo Linux, trabajando fundamentalmente en una [máquina virtual](#) disponible a través de la infraestructura [IaaS](#) de la ULL. Es por ello que resulta muy conveniente que el alumnado tenga instalado Linux en un ordenador personal con el que trabaje desde su casa. Se optará preferentemente por la distribución Ubuntu para que sea la misma que tiene la máquina virtual (VM) de la asignatura. Hay al menos tres opciones para ello, que son las siguientes en orden de mayor a menor idoneidad para el trabajo en la asignatura:
 - Si dispone Ud. de un ordenador propio, de uso exclusivo, que pueda formatear (borrando por tanto toda la información) instale directamente Ubuntu en él siguiendo (por ejemplo) [estas instrucciones](#). Para esta instalación necesitará Ud. crear un pendrive desde el que pueda arrancar el ordenador. Siga para ello (por ejemplo) [estas otras instrucciones](#).
 - Instalar Ubuntu como un sistema “invitado” dentro de Windows usando para ello un software de virtualización como VirtualBox. La página [Install Ubuntu on Oracle VirtualBox](#) contiene las instrucciones a seguir para instalar Ubuntu como sistema invitado en Windows.
 - Utilizar WSL, [Windows subsystem for Linux](#). WSL es una característica introducida en Windows 10 que permite instalar un núcleo Linux directamente sobre el sistema operativo de Microsoft.

Las tres opciones anteriores están ordenadas atendiendo a la que consideramos idoneidad de cada una, siendo la primera la más idónea, pero este orden posiblemente coincide con el orden inverso de complejidad de la instalación: la primera es posiblemente más compleja que la tercera. Tenga esto en cuenta a la hora de decidirse por cualquiera de estas opciones.

Una opción alternativa que se considera menos adecuada consiste en no instalar un sistema Linux sino acceder a la VM IaaS de la asignatura desde Windows usando para ello un cliente `ssh`. Se recomienda para este caso instalar en Windows el [cliente ssh PuTTY](#) que puede Ud. descargar libremente. [Este vídeo](#) muestra cómo configurar la VPN en Windows y cómo usar Putty para acceder a una VM.

En todo caso recuerde que si desea acceder a las máquinas de la Universidad desde fuera del campus universitario necesitará Ud. configurar una conexión usando [VPN](#). Para configurar la conexión VPN siga las instrucciones de la página [Servicio de VPN de la ULL](#). Para conexiones VPN usando Windows ha de instalar la aplicación FortiClient tal como se explica en el documento [Guía de configuración del cliente VPN FortiClient. Sistema Operativo Windows](#). La imagen de la Figura 1 muestra el establecimiento de la conexión VPN con la red de la ULL; la Figura 2 muestra el inicio de sesión, mientras que la Figura 3 muestra la conexión ya establecida.

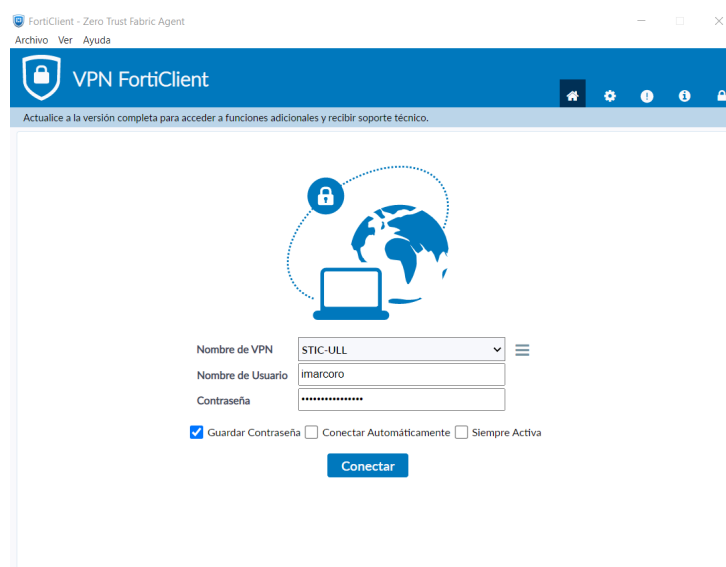


Figura 1: Establecimiento de una conexión vpn usando FortiClient

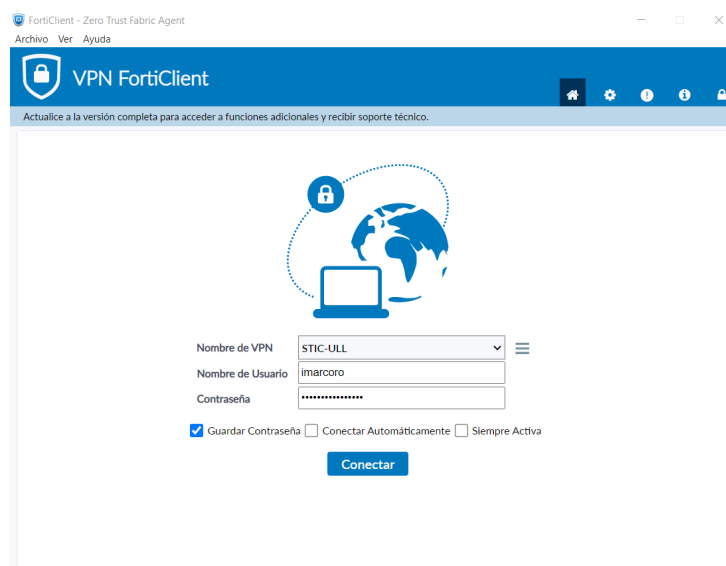


Figura 2: Inicio de sesión con FortiClient

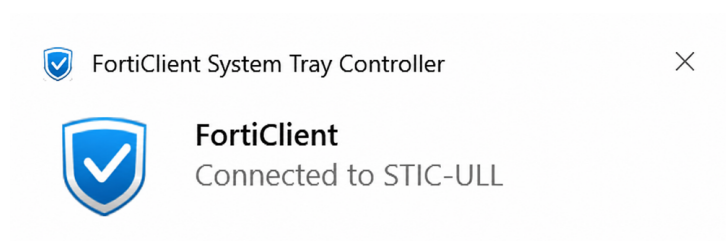


Figura 3: Conexión vpn establecida con FortiClient

[Este vídeo](#) muestra cómo establecer en linux una conexión `vpn` y [este otro](#) muestra cómo conectar desde linux usando `ssh` con una VM del IaaS.

Dirección de correo alternativa

2. Acceda al [portal de gestión de usuarios](#) del Servicio TIC de la ULL y configure allí una dirección de correo electrónico alternativa a su dirección `aluXXXX@ull.edu.es`. Las direcciones (alias) alternativas que el sistema le ofrece han de incluir necesariamente dos dígitos numéricos (sin significado específico alguno) y permiten que su dirección de correo sea más legible y fácil de recordar, sobre todo para otras personas. Podrá utilizar indistintamente las direcciones `aluXXXX@ull.edu.es` y el alias que configure.

GitHub

3. [GitHub](#) es una plataforma de desarrollo colaborativo para alojar proyectos utilizando el sistema de control de versiones Git. Cree una cuenta en GitHub, si no dispone de una. Configure el perfil de esa cuenta de modo que incluya una imagen (fotografía) en la que se le reconozca y haga que la cuenta de e-mail asociada sea la dirección institucional o su alias. Para la configuración de esa cuenta se le recomienda usar su nombre real, puesto que sus repositorios de código en GitHub pasarán a formar parte de su curriculum profesional.

El editor Vim

4. Si desea utilizar el editor `vim` estudie los primeros pasos de [este tutorial](#) para que aprenda lo básico sobre cómo modificar un fichero usando `vi`. Con el estudio de este otro [tutorial interactivo on-line](#) puede aprender lo mínimo que necesita para usar `vi` en esta sesión. Si está Ud. interesada/o en aprender `vim` (es un editor muy potente pero los comienzos son difíciles) dispone de muchos tutoriales en la web. [Este tutorial](#) explica lo básico de `vim` a través de ejemplos concretos con ficheros de texto. Para editar algunas líneas concretas de un fichero de texto usando `vi` siga estas indicaciones:
 - a) Use las teclas con flechas arriba/abajo para mover el cursor a la línea que desee editar.
 - b) Antes de modificar el texto ha de presionar `i` para acceder al modo de inserción de `vim`.
 - c) Cuando acabe de modificar el texto, pulse `ESC` (para salir del modo de inserción)

- d) Ahora escriba `:wq!` y presione **ENTER** para guardar los cambios en disco. W es para escribir (Write), Q para salir (Quit) y ! se usa para forzar la escritura.

El Entorno ULL-IaaS

5. Tenga en cuenta que el acceso a la infraestructura IaaS está ligado a que esté Ud. registrada/o en el Aula Virtual de la Asignatura. Siga las instrucciones de ese documento para acceder a la [interfaz web de las máquinas IaaS](#).
6. Inicie sesión en Linux en alguno de los PCs de una sala del Centro de Cálculo o bien desde otro ordenador. Recuerde tener iniciada una conexión **vpn** si trabaja desde fuera del campus universitario. En este documento se denominará máquina remota a la VM del **IaaS-ULL** y máquina local al PC en el que está Ud. trabajando.
7. Acceda a la [interfaz web](#) de la plataforma IaaS-ULL y autentíquese en esa interfaz con sus credenciales (username + password) de la cuenta institucional. La imagen de la Figura 4 muestra la pantalla de acceso a la interfaz. Vea el estado de la máquina y arránquela para comenzar a trabajar con ella.

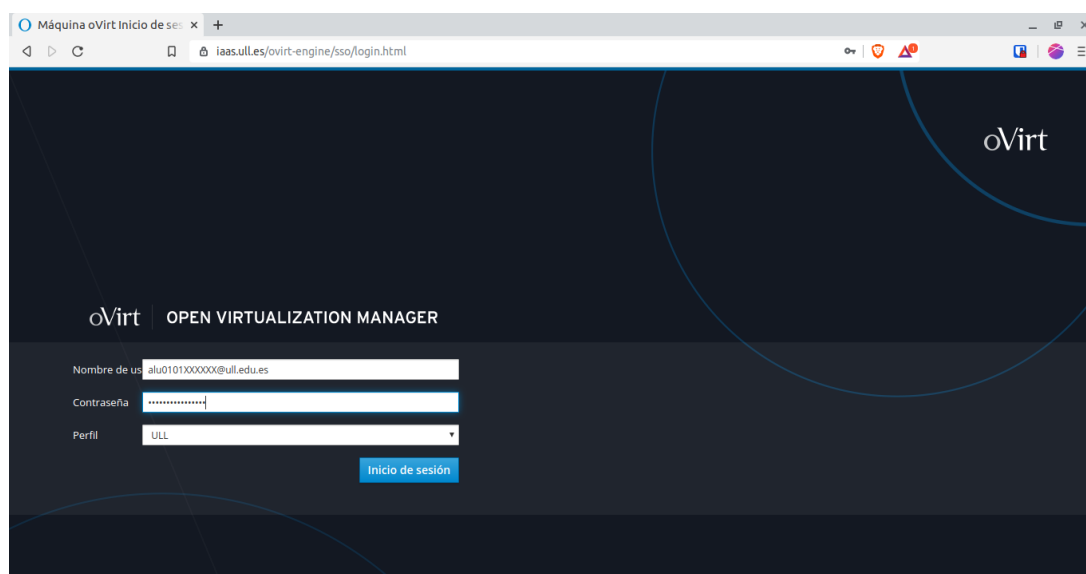


Figura 4: Acceso a la interfaz web del Servicio IaaS

8. La imagen de la Figura 5 muestra las VMs disponibles. Inicialmente la máquina de la asignatura aparece con estado “Apagado” y se arranca mediante el botón “Ejecutar”. El proceso de arranque de la máquina puede durar unos minutos.
9. Una vez que la máquina haya arrancado, tome nota de la Dirección IP de la máquina, que se muestra en el apartado “Detalles” de la máquina. La imagen de la Figura 6 muestra la pantalla de información correspondiente a una VM y en ella se muestra la dirección IP de la máquina. La [dirección IP](#) es una secuencia de números (de la forma

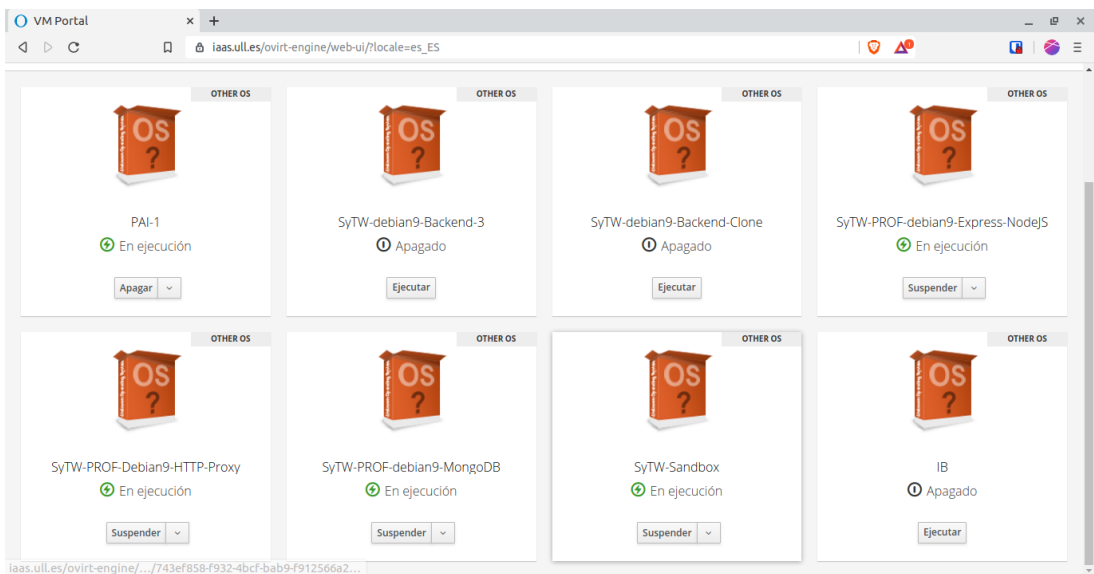


Figura 5: Máquinas virtuales disponibles para un usuario del servicio

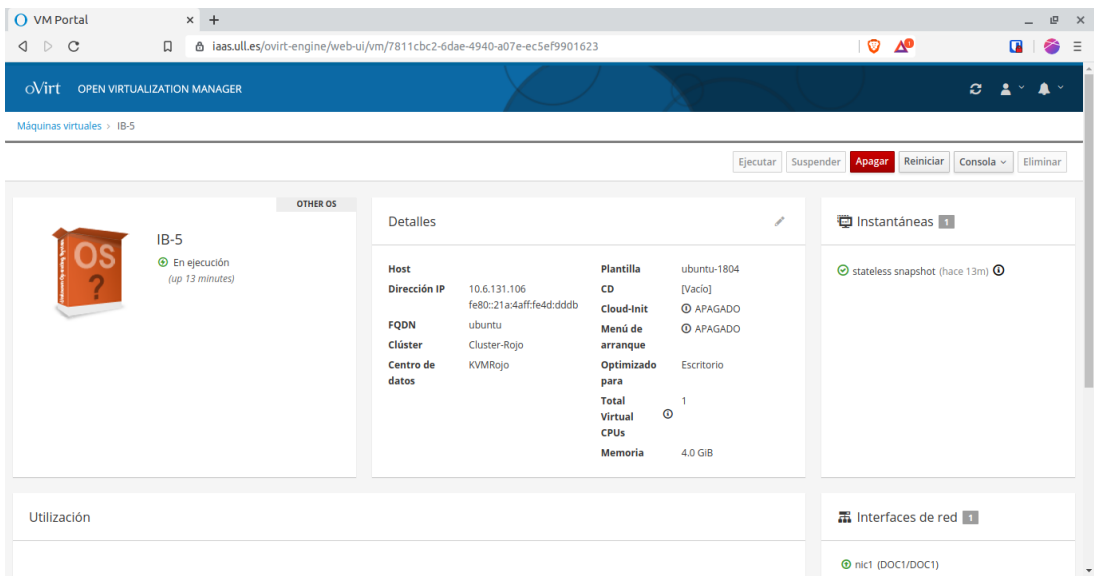


Figura 6: Información de la VM IB-5

10.6.131.106) que identifican de forma unívoca a cualquier dispositivo conectado a Internet. Esta dirección será necesaria para establecer conexiones directas a la máquina a través de `ssh` desde su casa o desde las salas del Centro de Cálculo de la ESIT. Anote esa dirección IP puesto que la máquina conserva esa dirección IP de forma estable. Si en algún momento experimenta dificultades de conexión, conecte a través de la interfaz web y compruebe que la dirección de la máquina no ha cambiado. Para consultar la IP de una máquina en un terminal Linux utilice el comando:

```
$ ifconfig -a
```

10. Abra una conexión `ssh` con su máquina usando la dirección IP de la misma que ha obtenido en el paso anterior. Si esa conexión la realiza desde Linux el comando que ha de usar para establecerla es

```
$ ssh usuario@10.6.131.106
```

Cambiando en el comando anterior, la dirección IP por la de su máquina. En cuanto se conecte a la máquina remota, el sistema le pedirá sus credenciales. Recuerde que inicialmente esas credenciales de acceso son Username: `usuario` y password: `usuario`.

Preste mucha atención a este paso porque si comete un error, puede ser irreparable y su máquina resultará inaccesible. En este primer acceso el sistema le solicitará que introduzca la contraseña actual y que escriba dos veces la nueva contraseña elegida. No se preocupe por ahora por la contraseña que elija puesto que siempre la puede cambiar en el futuro pero anote el password que elija para no perderlo u olvidarlo. La recomendación es que elija ahora un password simple (algo como 12345 o similar). Ya en el futuro podrá cambiarlo por otro más robusto. Compruebe a continuación el sistema operativo y versión del mismo:

```
$ lsb_release -a
```

11. Actualice el software (paquetes) de la máquina siguiendo las indicaciones de [esta página](#) (por ejemplo). Los comandos que tendrá que utilizar son:

```
$ sudo apt update
$ sudo apt upgrade
$ sudo apt autoremove
```

la imagen de la Figura 7 muestra el resultado de ejecución del primero de estos comandos y la Figura 8 el del segundo de ellos. Cuando el sistema le pregunte si hacerlo, elija `No instalar grub`. Los comandos que se ejecutan con `sudo` se están ejecutando como “superusuario” de modo que nunca ejecute comandos con `sudo` salvo que sepa muy bien lo que está haciendo o se le indique (como en este caso) cómo usarlo.

12. Edite los ficheros necesarios para [cambiar el nombre lógico de la máquina](#) que le

```
Archivo Editar Ver Buscar Terminal Ayuda
usuario@ubuntu:~$ sudo apt update
[sudo] contraseña para usuario:
Obj:1 http://security.ubuntu.com/ubuntu bionic-security InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu bionic InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu bionic-updates InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu bionic-backports InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias
Leyendo la información de estado... Hecho
Se pueden actualizar 167 paquetes. Ejecute «apt list --upgradable» para verlos.
usuario@ubuntu:~$
```

Figura 7: Resultado de la ejecución del comando `apt update`

```
Archivo Editar Ver Buscar Terminal Ayuda
Desempaquetando curl (7.58.0-2ubuntu3.9) sobre (7.58.0-2ubuntu3.7) ...
Preparando para desempaquetar .../067-libcurl4_7.58.0-2ubuntu3.9_amd64.deb ...
Desempaquetando libcurl4:amd64 (7.58.0-2ubuntu3.9) sobre (7.58.0-2ubuntu3.7) ...
Preparando para desempaquetar .../068-libdevmapper-event1.02.1_2k3a1.02.145-4.1ubuntu3.18.04.3_amd64.deb ...
Desempaquetando libdevmapper-event1.02.1:amd64 (2:1.02.145-4.1ubuntu3.18.04.3) sobre (2:1.02.145-4.1ubuntu3.18.04.1) ...
Preparando para desempaquetar .../069-liblvm2cmd2.02.2.02.176-4.1ubuntu3.18.04.3_amd64.deb ...
Desempaquetando liblvm2cmd2.02:amd64 (2.02.176-4.1ubuntu3.18.04.3) sobre (2.02.176-4.1ubuntu3.18.04.1) ...
Preparando para desempaquetar .../070-dmeventd_2k3a1.02.145-4.1ubuntu3.18.04.3_amd64.deb ...
Desempaquetando dmeventd (2:1.02.145-4.1ubuntu3.18.04.3) sobre (2:1.02.145-4.1ubuntu3.18.04.1) ...
Preparando para desempaquetar .../071-gtk-update-icon-cache_3.22.30-1ubuntu4_amd64.deb ...
Desempaquetando gtk-update-icon-cache (3.22.30-1ubuntu4) sobre (3.22.30-1ubuntu3) ...
Preparando para desempaquetar .../072-python3-twisted-bin_17.9.0-2ubuntu0.1_amd64.deb ...
Desempaquetando python3-twisted-bin:amd64 (17.9.0-2ubuntu0.1) sobre (17.9.0-2) ...
Preparando para desempaquetar .../073-python3-twisted_17.9.0-2ubuntu0.1_all.deb ...
Desempaquetando python3-twisted (17.9.0-2ubuntu0.1) sobre (17.9.0-2) ...
Preparando para desempaquetar .../074-landscape-common_18.01-0ubuntu3.5_amd64.deb ...
Desempaquetando landscape-common (18.01-0ubuntu3.5) sobre (18.01-0ubuntu3.3) ...
Preparando para desempaquetar .../075-libcups2_2.2.7-1ubuntu2.8_amd64.deb ...
Desempaquetando libcups2:amd64 (2.2.7-1ubuntu2.8) sobre (2.2.7-1ubuntu2.6) ...
Preparando para desempaquetar .../076-libcurl3-gnutls_7.58.0-2ubuntu3.9_amd64.deb ...
Desempaquetando libcurl3-gnutls:amd64 (7.58.0-2ubuntu3.9) sobre (7.58.0-2ubuntu3.7) ...
Preparando para desempaquetar .../077-liblvm2app2.2.2.02.176-4.1ubuntu3.18.04.3_amd64.deb ...
Desempaquetando liblvm2app2.2:amd64 (2.02.176-4.1ubuntu3.18.04.3) sobre (2.02.176-4.1ubuntu3.18.04.1) ...
Preparando para desempaquetar .../078-lbsasl2-modules_2.1.27-101-g0780600+dfsg-3ubuntu2.1_amd64.deb ...
Desempaquetando libsasl2-modules:amd64 (2.1.27-101-g0780600+dfsg-3ubuntu2.1) sobre (2.1.27-101-g0780600+dfsg-3ubuntu2) ...
Preparando para desempaquetar .../079-libtiff5_4.0.9-Subuntu0.3_amd64.deb ...
Desempaquetando libtiff5:amd64 (4.0.9-Subuntu0.3) sobre (4.0.9-Subuntu0.2) ...
Preparando para desempaquetar .../080-libxslt1.1_1.1.29-Subuntu0.2_amd64.deb ...
Desempaquetando libxslt1.1:amd64 (1.1.29-Subuntu0.2) sobre (1.1.29-Subuntu0.1) ...
Preparando para desempaquetar .../081-linux-firmware_1.173.19_all.deb ...
Desempaquetando linux-firmware (1.173.19) sobre (1.173.8) ...
Seleccionando el paquete linux-modules-4.15.0-112-generic previamente no seleccionado.
Preparando para desempaquetar .../082-linux-modules-4.15.0-112-generic_4.15.0-112.113_amd64.deb ...
Desempaquetando linux-modules-4.15.0-112-generic (4.15.0-112.113) ...

Progreso: [ 59%] [#####]
```

Figura 8: Resultado de la ejecución del comando `apt upgrade`

ha sido asignada. Se propone utilizar como nombre algo como Ubuntu-18-ASIG-XXX (cambiando “ASIG” por el acrónimo de la asignatura y “XXX” por lo que Ud. quiera), aunque puede Ud. usar para su máquina el nombre que prefiera. Para realizar ese cambio ha de editar Ud. los siguientes ficheros (necesita usar **sudo** para tener permisos de root al tratarse de ficheros del sistema) y sustituir en ellos el nombre actual de la máquina (que es “ubuntu”) por el que haya elegido:

```
$ sudo vi /etc/hostname
$ sudo vi /etc/hosts
```

Para que este cambio tenga efecto, ha de reiniciar la máquina. Siempre puede reiniciar la máquina desde la interfaz web de administración o bien usando el comando:

```
$ sudo reboot
```

Git y GitHub

13. Instale git en su máquina:

```
$ sudo apt install git
```

y compruebe que está instalado:

```
$ git --version
```

14. Consiga que se pueda subir código desde su VM hacia su cuenta GitHub sin necesidad de autenticación. Consulte para ello las instrucciones [Adding a new SSH key to your GitHub account](#)
15. Cree un directorio practicas y clone en él un repositorio git:

```
cd
mkdir practicas
cd practicas
git clone git@github.com:fsande/IB-P01-EntornoIaaS.git
```

Visual Studio Code (VSC)

16. En la máquina local ejecute el VSC y siga [estas instrucciones](#) para configurar la edición remota de ficheros alojados en la VM. Para instalar VSC en la instalación Linux de su casa siga [estas instrucciones](#) descargando el paquete `.deb`. Resulta útil realizar el paso 3 (opcional) del apartado “*SSH host setup*” que se muestra en [esta página](#). Ejecute para ello (sustituyendo la dirección IP por la de su VM):

```
$ ssh-keygen -t rsa -b 4096
$ export USER_AT_HOST=usuario@10.6.131.106
$ export PUBKEYPATH="$HOME/.ssh/id_rsa.pub"
$ ssh-copy-id -i "$PUBKEYPATH" "$USER_AT_HOST"
```

La imagen de la Figura 9 muestra el paso 2 del apartado “Connect to a remote host” de esas instrucciones mientras que la de la Figura 10 muestra la ventana de autenticación de VSC para darle acceso a la VM. [Este vídeo](#) ilustra cómo configurar la edición remota de ficheros alojados en máquinas IaaS-ULL usando VSC.

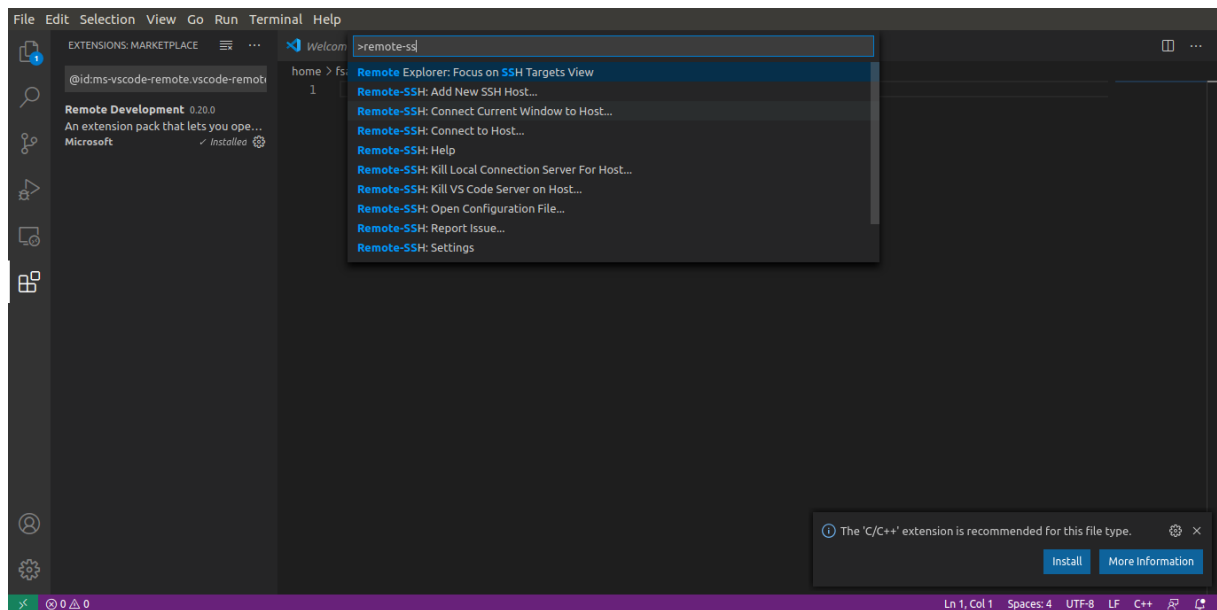


Figura 9: Conexión a una máquina remota en VSC

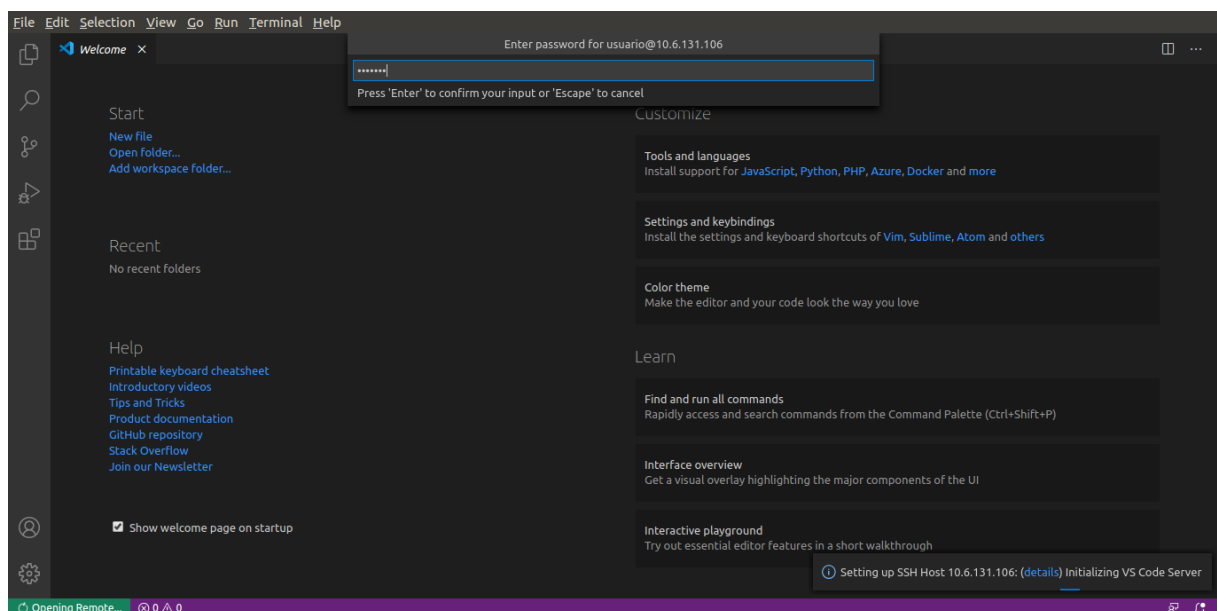


Figura 10: Autenticación en la máquina remota accedida desde VSC

17. Utilice el VSC para escribir el código fuente del programa `hello_world.cc`. Grabe ese fichero en el directorio `practicas` creado en su VM. Acceda a la VM usando `ssh`, compile y ejecute ese programa. Para compilar el programa escriba el comando:

```
$ g++ -std=c++14 -g -Wall -o hello_world hello_world.cc
```

Y para ejecutarlo escriba:

```
$ ./hello_world
```

Simplificar la conexión a la VM

18. Siga [estas instrucciones](#) para establecer la configuración de la máquina de modo que se pueda conectar a ella sin necesidad de escribir el password en cada conexión. Para poder conectarse por `ssh` con las VMs de IaaS ULL ha de autenticarse en la página <http://acceso.ull.es/>. Para trabajar en las sesiones de evaluación de prácticas, comience por abrir un navegador y autenticarse en esa página. Recuerde que en caso de acceder desde fuera de del campus ha de hacerlo mediante una conexión VPN. Consulte esta referencia (en el Centro de Cálculo, por ahora no lo necesita) para conectarse a través de `vpn`.

También resulta conveniente utilizar alguno de los métodos (`ssh config` o alias) que se presentan en [estas instrucciones](#) de modo que se simplifique la conexión con la máquina remota pudiendo escribir algo como:

```
$ ssh vm-asignatura
```

para conectarse a la VM.

4. Estilo y formato del código

Esta relación no pretende ser exhaustiva. Contiene una serie de requisitos y/o recomendaciones cuyo cumplimiento se impulsará a la hora de evaluar las prácticas de la asignatura.

- Estudien la finalidad de los especificadores de visibilidad (`public`, `private`, `protected`) de atributos y métodos. Una explicación breve y simple es ésta [5] aunque es fácil hallar muchas otras. Se debe restringir los atributos y métodos públicos a aquellos que son estrictamente necesarios para que un programa cliente utilice la clase en cuestión.
- El principio de responsabilidad única (*Single responsibility principle*, *SRP*) [6] es uno de los conocidos como principios “SOLID” y es posiblemente uno de los más fáciles de comprender. Dejando a un lado el resto de principios SOLID, que pueden

ser más complejos, merece la pena que lean algo sobre el SRP. Ese principio es una de las razones por las que en sus programas no debiera haber una única clase “*que se encarga de todo*”. En esta referencia [7] tienen un sencillo ejemplo (el de la clase `Person` que puede servir para ilustrar el SRP, pero no es difícil encontrar múltiples ejemplos del SRP. Tengan en cuenta que es un principio de orientación a objetos, y por lo tanto independiente del lenguaje de programación: pueden encontrar ejemplos de código en C++, Java, Python u otros lenguajes.

- Un programa simple en C++ debiera incluir al menos 3 ficheros:
 - `X.h` para la definición de la clase `X`
 - `X.cc` para la implementación de la clase `X`
 - `ClienteX.cc` para el programa “cliente” que utiliza la clase `X`
- Para evitar la doble inclusión de un mismo fichero (habitualmente un fichero de cabecera, *header*) en un código fuente hay básicamente dos posibilidades: las *include guards* [17] y el uso de la directiva `#pragma once` [18]. Es conveniente que conozcan ambas alternativas, el funcionamiento de cada una y que tengan en cuenta que la directiva `#pragma once`, aunque soportada por la mayor parte de los compiladores de C++, no es estándar. Para más información sobre este asunto pueden leer esta discusión de StackOverflow [19].
- Ejemplo de comentario de cabecera (inicial) para (todos) los ficheros de un proyecto (práctica) de la asignatura:

```
// Universidad de La Laguna
// Escuela Superior de Ingeniería y Tecnología
// Grado en Ingeniería Informática
// Asignatura: Computabilidad y Algoritmia
// Curso: 2º
// Práctica 2: Cadenas y lenguajes
// Autor: Nombre y Apellidos
// Correo: aluXXXXX@ull.edu.es
// Fecha: 16/09/2025
// Archivo cya-P02-strings.cc: programa cliente.
//         Contiene la función main del proyecto que usa las clases X e Y
//         para ... (indicar brevemente el objetivo)
// Referencias:
//         Enlaces de interés
//
// Historial de revisiones
//         16/09/2025 - Creación (primera versión) del código
```

- Además de la información anterior sobre el autor, la fecha y la asignatura, la cabecera también debería contener al menos una breve descripción sobre lo que hace el código incluyendo los objetivos del proyecto en general, las estructuras de datos utilizadas,

así como un listado de modificaciones (bug fixes) que se han ido introduciendo en el código. El fichero de cabecera sería básicamente el mismo para todos los ficheros (*.cc, *.h) del proyecto, con la salvedad de que varía la descripción del contenido del fichero y su finalidad.

- El utilizar una estructura de directorios con nombres estándar (bin, lib, src, config, etc.) para los proyectos de desarrollo de software es una buena práctica cuando se trata de proyectos que involucran un elevado número de ficheros. No obstante, para la mayoría de prácticas de esta asignatura, que no contienen más de una decena de ficheros, colocar todos los ficheros de cada práctica en un mismo directorio (con nombre significativo *CyA/practicas/p02_strings*, por ejemplo) es posiblemente más eficiente a la hora de localizar y gestionar los ficheros de cada una de las prácticas (proyectos).
- En el directorio de cada práctica, como se ha indicado anteriormente, debería haber un fichero **Makefile** de modo que el comportamiento del programa **make** fuera tal que al ejecutar:

```
make clean
```

el programa dejara en el directorio en que se ejecuta solamente los ficheros conteniendo código fuente (ficheros *.h y *.cc) y el propio **Makefile**. En estos breves tutoriales [3, 4] se explica de forma incremental cómo construir un fichero Makefile para trabajar con la compilación de proyectos simples en C++.

- A continuación enumeraremos algunas cuestiones relativas al formato del código:

1. A ambos lados de un operador binario han de escribir un espacio:

a + b

En lugar de:

a+b

2. SIEMPRE después de una coma, ha de ir un espacio.
3. Se debe indentar el código usando espacios y NO tabuladores. Cada nivel de indentación ha de hacerse con 2 espacios.
4. TODO identificador (de clase, de método, de variable, ...) ha de ser significativo. No se pueden usar identificadores de un solo caracter salvo para casos concretos (variable auxiliar para un bucle o similar).
5. TODO fichero de código de un proyecto ha de contener un prólogo con comentarios de cabecera donde se indique al menos: Autor, datos de contacto, Fecha, Asignatura, Práctica, Finalidad del código. Véase el ejemplo de cabecera proporcionado en este documento.

6. Asimismo todos los métodos y clases han de tener al menos un mínimo comentario que documente la finalidad del código.
7. No comentar lo obvio. No se trata de comentar por comentar, sino de aclarar al lector la finalidad del código que se escribe [15].
8. Como regla de carácter general, el código fuente de los programas no debiera contener de forma explícita constantes. En lugar de escribir

```
double balance[10];
```

Es preferible:

```
const int SIZE_BALANCE = 10;
...
double balance[SIZE_BALANCE];
```

Para el caso de las constantes numéricas, las únicas excepciones a esta regla son los valores 0 y 1. Tengamos en cuenta que las constantes no son solamente numéricas: puede haber constantes literales, de carácter o de otros tipos. Así en lugar de escribir:

```
myString.find(' ');
```

Es preferible:

```
const char SPACE = ' ';
...
myString.find(SPACE);
```

9. Cuando se programa un código encadenando múltiples sentencias if-else, ello suele ser síntoma de una mala práctica y, en efecto, debería buscarse alguna alternativa. En este sentido les recomendamos que revisen las diferentes sugerencias que se realizan al hilo de esta discusión [16].
10. Como regla general, se espera que todo el código fuente siga la guía de estilo de Google para C++ [14]. Dentro de esa guía, y para comenzar, prestaremos particular atención a los siguientes aspectos:
 - Formateo del código (apartado *Formatting* en [14])
 - Comentarios de código de diverso tipo (apartado *Comments* en [14])
 - Nominación de identificadores, clases, ficheros, etc. (apartado *Naming* en [14])

Referencias

- [1] Números de Pell: https://es.wikipedia.org/wiki/N%C3%BAmero_de_Pell
- [2] Código ASCII: <https://es.wikipedia.org/wiki/ASCII>
- [3] Makefile Tutorial: <http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor>
- [4] C++ Makefile Tutorial: <https://makefiletutorial.com>
- [5] Difference between private, public and protected inheritance: <https://stackoverflow.com/questions/860339/difference-between-private-public-and-protected-inheritance>
- [6] Single-Responsability Principle (SRP): https://en.wikipedia.org/wiki/Single_responsibility_principle
- [7] Example of the SRP: <https://stackoverflow.com/questions/10620022/what-is-an-example-of-the-single-responsibility-principle>
- [8] STL sets, <http://www.cplusplus.com/reference/set/set>
- [9] STL sets tutorial, https://www.w3schools.com/cpp/cpp_sets.asp
- [10] Using the C++ Standard Template Libraries, <https://link.springer.com/book/10.1007%2F978-1-4842-0004-9>
- [11] Sobrecarga de operadores, <https://es.cppreference.com/w/cpp/language/operators>
- [12] C++ Operator Overloading Example, <https://www.programiz.com/cpp-programming/operator-overloading>
- [13] Input/Output Operators Overloading in C++, https://www.tutorialspoint.com/cplusplus/input_output_operators_overloading.htm
- [14] Google C++ Style Guide, <https://google.github.io/styleguide/cppguide.html>
- [15] Commenting code: <https://www.cs.utah.edu/~germain/PPS/Topics/commenting.html>
- [16] If-else-if chains or multiple-if: <https://stackoverflow.com/questions/25474906/which-is-better-if-else-if-chain-or-multiple-if>
- [17] Include guard: https://en.wikipedia.org/wiki/Include_guard
- [18] Pragma once: https://en.wikipedia.org/wiki/Pragma_once
- [19] Pragma once vs. Include guards: <https://stackoverflow.com/questions/1143936/pragma-once-vs-include-guards>